Innovation Ways

TECHNICAL DOCUMENT

ARCHITECTURE & ENGINEERING

IW AI Core — Complete Architecture & End-to-End Flow

Standalone multi-project AI orchestration platform

VERSION	DATE	AUTHOR	AUDIENCE
1.2.0	2026-04-07	Sergio G. + Claude	Engineering / Architecture

Innovation Ways · innovation-ways.com

IW-AI-CORE-001

IW AI Core - Complete Architecture & End-to-End Flow

Project: IW AI Core (Innovation Ways AI Orchestration Platform)
Author: Sergio G. + Claude
Date: 2026-04-07
Status: Architecture Blueprint

1. Executive Summary

IW AI Core is a standalone platform that centralizes AI-assisted development orchestration across multiple software projects. It replaces the current file-based tracking system (markdown ID files, JSON manifests) embedded inside InnoForge with a database-backed, multi-project platform.

The core problem: The current system is tightly coupled to InnoForge. ID allocation uses markdown files with no locking (causing race conditions under concurrency). Execution state lives in JSON manifests scattered across directories. The dashboard, daemon, and executor scripts all live inside InnoForge's repository, making it impossible to reuse them for other projects.

The solution: Extract everything into a standalone repository (`iw-ai-core`) with a PostgreSQL database as the single source of truth for all operational state. Projects register themselves and the platform manages their entire AI workflow lifecycle from a unified dashboard.

Key Decisions (from user)

DECISION	RESOLUTION
Migration approach	Full migration, fresh start. No history import.
Extraction scope	All at once. Shutdown current, delete files, move to iw-ai-core.
Feature baseline	Current dashboard and executor are stable. Migrate as-is, evolve independently.
Skills distribution	Installable packages. Master version in iw-ai-core, "clones" installed per project.
Content storage	Two-tier model: design docs + reports in DB (always viewable, searchable); full artifacts compressed to archive (on-demand extraction).

2. System Architecture Overview

See Diagram 1: System Architecture Overview

2.1. Physical Layout

```
Developer Machine (iw-dev-01)
|
+-- /home/sergiog/dev/iw-ai-core/          <-- PLATFORM (new standalone repo)
|   +-- PostgreSQL (port 5433)              <-- ALL state for ALL projects
|   +-- Dashboard (port 9900)               <-- Unified web UI
|   +-- Daemon (1 process)                  <-- Orchestrates ALL projects
|   +-- Executor scripts                    <-- Project-agnostic bash
|   +-- iw CLI (pip install)                <-- Agent-to-DB bridge
|   +-- Base skills & templates             <-- Master copies
|   +-- archive/                            <-- Compressed artifacts (NOT in git)
|       +-- innoforge/I001.tar.zst          <-- Tier 2 storage per work item
|       +-- project-b/F001.tar.zst
|
+-- /home/sergiog/dev/iw-doc-plan/          <-- PROJECT A: InnoForge
|   +-- main/iw-doc-plan/                   <-- Main clone (agents, merges)
|   |   +-- .iw-orch.json                   <-- Project registration
|   |   +-- .claude/skills/                 <-- InnoForge-specific + synced base skills
|   |   +-- ai-dev/design/active/           <-- Only in-flight items (no done/ folder)
|   |   +-- ai-dev/workflow.md              <-- Workflow definition
|   |   +-- .worktrees/                     <-- Execution worktrees (daemon creates)
|   +-- development/iw-doc-plan/            <-- Dev clone (human coding)
|       +-- .claude/skills/                 <-- Same skills (synced)
|
+-- /home/sergiog/dev/project-b/            <-- PROJECT B: Future project
|   +-- .iw-orch.json
|   +-- .claude/skills/
|   +-- ai-dev/design/
|   +-- ai-dev/workflow.md
|
+-- /home/sergiog/dev/project-c/            <-- PROJECT C: Future project
    +-- .iw-orch.json
    +-- .claude/skills/
    +-- ai-dev/design/
    +-- ai-dev/workflow.md
```

2.2. What Lives Where

COMPONENT	LOCATION	WHY
Operational state (IDs, statuses, manifests, step results)	PostgreSQL in iw-ai-core	Atomic operations, queryable, no race conditions
Design docs & reports (Tier 1 — always viewable)	PostgreSQL `TEXT` columns	Instant rendering in dashboard, full-text search, RAG-ready
Full artifacts (Tier 2 — on-demand)	Compressed archives in `iw-ai-core/archive/`	Prompts, evidences, logs, screenshots; extracted to tmp on request
Active work items (in-flight)	Each project's `ai-dev/design/active/`	Agents need file access during execution
Workflow definition (steps, agents, timeouts)	Each project's `ai-dev/workflow.md`	Evolves with the project code
Skills (Claude Code skills)	Master in iw-ai-core, clones in each project	Versioned distribution, project overrides
Daemon + executor + dashboard	iw-ai-core repo	Shared infrastructure, project-agnostic
Project config	`.iw-orch.json` in each project	Self-describing registration

Key principle: Active work items live in the project repo while agents work on them. Once completed, `iw archive` stores searchable content (design doc, reports) in the DB and compresses everything else to the archive directory. The project repo stays lean — no `done/` folder accumulating hundreds of items.

---

3. End-to-End Flow: Creating a New Incident

See Diagram 2: End-to-End Flow

This is the complete journey from typing `/iw-new-incident` in Claude Code to a merged fix on main. Every step is explained in detail.

Phase 1: Work Item Creation (Human + Claude Code)

Context: Developer is working in the InnoForge development clone (`/home/sergiog/dev/iw-doc-plan/development/iw-doc-plan`). They've found a bug and want to create an incident.

Step 1.1: Skill Invocation

```
Developer types: /iw-new-incident
```

Claude Code loads the skill from `.claude/skills/iw-new-incident/SKILL.md`. This skill was synced from iw-ai-core via `iw sync-skills` and contains instructions that tell Claude how to create an incident.

Step 1.2: Atomic ID Allocation

The skill instructs Claude to run:

```
iw next-id --type incident --project innoforge
# Returns: I001
```

What happens under the hood:

1. The `iw` CLI (installed globally via `pip install -e /path/to/iw-ai-core`) connects to PostgreSQL at `localhost:5433`
2. Executes an atomic query:
   `sql BEGIN; SELECT next_number FROM id_sequences WHERE project_id = 'innoforge' AND prefix = 'I' FOR UPDATE; -- Row lock prevents concurrent reads UPDATE id_sequences SET next_number = next_number + 1 WHERE project_id = 'innoforge' AND prefix = 'I'; COMMIT; -- Returns: I001`
3. The `FOR UPDATE` lock guarantees that even if 10 agents call `iw next-id` simultaneously, each gets a unique sequential ID. **No race conditions possible.**

Step 1.3: Design Document Creation

Claude creates the complete design package:

```
ai-dev/design/active/I001/
+-- I001_Issue_Design.md            <-- Design document
+-- workflow-manifest.json          <-- Step definitions (read-only reference)
+-- prompts/
|   +-- I001_S01_Backend_prompt.md    <-- Implementation prompt
|   +-- I001_S02_CodeReview_Backend_prompt.md
|   +-- I001_S03_CodeReview_Final_prompt.md
|   +-- I001_S04_QualityValidation_prompt.md
+-- evidences/
    +-- pre/                        <-- Bug screenshots, logs
```

These files are created in the **InnoForge repo** (development clone), not in iw-ai-core. Content stays with the project.

Step 1.4: Work Item Registration

The skill instructs Claude to register the work item in the database:

```
iw register I001 "Fix template rendering timeout" \
  --project innoforge \
  --type incident \
  --design-doc ai-dev/design/active/I001/I001_Issue_Design.md \
  --steps-from ai-dev/design/active/I001/workflow-manifest.json
```

What happens:

1. CLI connects to PostgreSQL
2. Inserts into `work_items` table:
   `sql INSERT INTO work_items (id, project_id, type, title, status, phase, design_doc_path) VALUES ('I001', 'innoforge', 'Issue', 'Fix template rendering timeout', 'draft', 'active', 'ai-dev/design/active/I001/I001_Issue_Design.md') ON CONFLICT DO NOTHING;`
3. Inserts all workflow steps into `workflow_steps` table (parsed from manifest)
4. Idempotent: if called twice, no duplicate

Critical safety: Even if Claude's session crashes between step 1.3 and 1.4, the skill's error handling ensures `iw register` is called. The design doc files exist on disk, and the DB entry is created. Nothing is lost.

Phase 2: Human Review & Approval

Step 2.1: Review in Dashboard

Developer opens `http://localhost:9900/project/innoforge/queue`:

- Sees I001 in the "Pending Review" section
- Clicks to view: design document rendered as HTML, all prompts listed, step pipeline visualized
- Reviews the design for correctness

Step 2.2: Approval

From the dashboard, developer clicks “Approve” button. Or from CLI:

```
iw approve I001 --project innoforge
```

What happens:

```
UPDATE work_items SET status = 'approved', updated_at = now()
  WHERE project_id = 'innoforge' AND id = 'I001' AND status = 'draft';
```

The item is now ready for execution. It appears in the dashboard’s “Ready for Execution” queue.

Phase 3: Batch Planning & Launch

Step 3.1: Batch Creation

Developer wants to execute I001 along with other items. From Claude Code:

```
/iw-batch-execute I001 I002 I003 --max=4
```

Or from the dashboard: select items checkboxes, click “Create Batch”.

What happens:

1. `iw` CLI allocates batch ID: `BATCH-001`
2. Analyzes dependencies between items (from `work_items.depends_on`)
3. Creates execution groups (independent items can run in parallel)
4. Inserts into `batches` and `batch_items` tables
5. Status: `planning`

Step 3.2: Human Approval & Launch

```
make batch-launch BATCH=BATCH-001
# Or from dashboard: click "Launch" on batch detail page
```

What happens:

1. Prompts for confirmation (human must explicitly approve)
2. Updates batch status: `planning` -> `approved`
3. Daemon picks up the batch on its next poll cycle

Phase 4: Daemon Execution

See Diagram 3: Daemon Execution Pipeline

Step 4.1: Daemon Discovery

The daemon runs as a single Python process, polling every 60 seconds:

```
# Simplified daemon main loop
while running:
    for project in enabled_projects:
        active_batches = db.query(Batch).filter(
            project_id=project.id,
            status='approved' or status='executing'
        )
        for batch in active_batches:
            process_batch(project, batch)

    quota_monitor.poll_if_due()
    sleep(60)
```

The daemon finds BATCH-001 for InnoForge with status `approved`. It transitions to `executing`.

Step 4.2: Worktree Setup

For each item in the current execution group, the daemon calls:

```
executor/worktree_setup.sh I001 /home/sergiog/dev/iw-doc-plan/main/iw-doc-plan
```

What the script does (deterministic bash, no LLM):

1. Creates git worktree: `.worktrees/I001/` with branch `agent/I001-fix-template-timeout`
2. Copies design files from `ai-dev/design/active/I001/` into the worktree
3. Installs dependencies (Python venv + npm)
4. Syncs platform skills into worktree's `.claude/skills/`
5. Writes `execution_brief.json` (read-only agent context):
   `json { "item_id": "I001", "project_id": "innoforge", "title": "Fix template rendering timeout", "design_doc": "ai-dev/design/active/I001/I001_Issue_Design.md", "steps": [{ "step_id": "S01", "agent_label": "Backend", "prompt_file": "ai-dev/design/active/I001/prompts/I001_S01_Backend_prompt.md", "status": "pending" }] }`
6. Updates DB: `batch_items.status = 'setting_up'` -> `'executing'`

Step 4.3: Agent Launch

The daemon launches the LLM agent:

```
cd .worktrees/I001 && opencode run '/execute I001'
```

Or with Claude Code:

```
cd .worktrees/I001 && claude -p '/execute I001'
```

The agent runs inside the worktree, which is a full checkout of the InnoForge repo. It has:
- The project's `CLAUDE.md` (full project context)
- The project's `.claude/skills/` (project-specific + synced base skills)
- The `execution_brief.json` (what to do)
- The design doc and prompts (how to do it)

Step 4.4: Agent Step Execution

The `/execute` skill (synced from iw-ai-core) orchestrates the workflow:

For each step in the execution brief:

1. **Start step**: Agent calls `iw step-start I001 --step S01`
2. DB updates: `workflow_steps.status = 'in_progress'`, records `started_at`
3. **Execute**: Agent reads the prompt file, delegates to specialist subagent
4. Example: `@backend-impl` reads `I001_S01_Backend_prompt.md`
5. Implements the fix following TDD
6. Writes report to `ai-dev/design/active/I001/reports/I001_S01_Backend_report.md`
7. **Complete step**: Agent calls `iw step-done I001 --step S01 --report path/to/report.md`
8. DB updates: `workflow_steps.status = 'completed'`, records report path and `completed_at`
9. **Code review**: Next step is `S02_CodeReview_Backend`
10. Same pattern: `iw step-start`, execute, `iw step-done`
11. If review finds issues: `iw step-fail I001 --step S02 --reason "3 HIGH findings"`
12. Triggers fix cycle (up to 5 attempts)
13. **Quality validation**: Final steps run lint, tests, type checks
14. Each gate is a separate step with its own pass/fail status
15. Failed gates trigger auto-fix cycles

Step 4.5: Agent-to-Database Communication

This is the key architectural pattern. Agents never write state files. They call the `iw` CLI which updates the database.

```
Agent (LLM in worktree)        iw CLI            PostgreSQL
      |                          |                  |
      |-- iw step-start I001 S01 ->                  |
      |                          |-- UPDATE steps SET  ->
      |                          |   status='in_progress'|
      |                          |<- OK               |
      |<- OK                     |                  |
      |                          |                  |
      | (does work, writes files) |                  |
      |                          |                  |
      |-- iw step-done I001 S01 -->                  |
      |   --report path.md       |-- UPDATE steps SET  ->
      |                          |   status='completed'  |
      |                          |   report_file=path    |
      |                          |<- OK               |
      |<- OK                     |                  |
```

Why CLI and not direct DB access?

- Agents are LLM sessions — they shouldn't have DB credentials
- CLI provides a stable, versioned interface
- CLI can validate inputs, enforce state transitions
- CLI is testable independently
- Works across any LLM tool (Claude Code, OpenCode, future tools)

Phase 5: Completion & Merge

Step 5.1: All Steps Complete

When the last step completes, the daemon detects it on its next poll:

```
SELECT COUNT(*) FROM workflow_steps
  WHERE project_id = 'innoforge' AND work_item_id = 'I001'
    AND status != 'completed';
-- Returns: 0 (all done)
```

Step 5.2: Merge to Main

The daemon runs the merge (one at a time per project):

```
executor/worktree_commit.sh I001 /home/sergiog/dev/iw-doc-plan/main/iw-doc-plan
```

1. Squash-merge worktree branch to main
2. Update DB: `batch_items.status = 'merged'`, `batch_items.merged_at = now()`
3. Clean up worktree

Step 5.3: Batch Completion

When all items in the batch are merged:
- Batch status: `completed`
- If `auto_publish = true`: daemon pushes to origin
- Dashboard shows success notification via SSE

Step 5.4: Archive (Two-Tier Storage)

Developer archives from dashboard or CLI:

```
iw archive I001 --project innoforge
```

What happens (Tier 1 — store searchable content in DB):

1. Read `I001_Issue_Design.md` content -> store in `work_items.design_doc_content`
2. Read each `*_report.md` -> store in `workflow_steps.report_content`
3. Generate `tsvector` for full-text search -> `work_items.design_doc_search`
4. Optionally: generate AI summary (2-3 lines) -> `work_items.summary`

What happens (Tier 2 — compress full artifacts to archive):

1. Create `archive/innoforge/I001.tar.zst` containing the full work item folder
2. Record `archive_path`, `archive_size_bytes`, `archived_at` in DB
3. Delete `ai-dev/design/active/I001/` from the project repo (project stays lean)

Result:
- Design doc and reports are always viewable in the dashboard (from DB)
- Full artifacts (prompts, evidences, logs) available on demand via archive extraction
- Project repo `ai-dev/design/active/` only contains in-flight work items
- No `done/` folder accumulating hundreds of items in the project repo

---

4. Multi-Project Management

See Diagram 4: Multi-Project Topology

This is how the platform manages 3 (or more) projects simultaneously from a single dashboard and daemon.

4.1. Project Registration

Each project self-describes with a `.iw-orch.json` file in its repo root:

```
{
  "project_id": "innoforge",
  "display_name": "InnoForge Document Platform",
  "repo_root": "/home/sergiog/dev/iw-doc-plan/main/iw-doc-plan",
  "dev_clone": "/home/sergiog/dev/iw-doc-plan/development/iw-doc-plan",
  "id_prefixes": { "Feature": "F", "Issue": "I", "ChangeRequest": "CR", "Batch": "BATCH" },
  "worktree_base": ".worktrees",
  "ai_dev_dir": "ai-dev",
  "cli_tool": "opencode",
  "quality_gates": ["ruff", "mypy", "pytest", "coverage", "semgrep"],
  "max_parallel": 4,
  "branch_prefix": "agent"
}
```

The central `projects.toml` in iw-ai-core links everything:

```
[[project]]
id = "innoforge"
path = "/home/sergiog/dev/iw-doc-plan/main/iw-doc-plan"
enabled = true

[[project]]
id = "project-b"
path = "/home/sergiog/dev/project-b"
enabled = true

[[project]]
id = "project-c"
path = "/home/sergiog/dev/project-c"
enabled = true
```

4.2. How the Dashboard Serves Multiple Projects

See Diagram 5: Dashboard Information Architecture

```
http://localhost:9900/                    <-- Project Selector (root)
|
|   +-----------------------------------------------+
|   | InnoForge          | 2 active batches, 15 items|
|   | Project B          | 1 batch, idle             |
|   | Project C          | No batches, 3 in queue    |
|   +-----------------------------------------------+
|
+-- /project/innoforge/                 <-- InnoForge Dashboard
|   +-- /batches                        <-- All InnoForge batches
|   +-- /batch/BATCH-001                    <-- Batch detail + timeline
|   +-- /queue                          <-- Pending items + designs
|   +-- /history                        <-- Completed items
|   +-- /analytics                      <-- Success rates, trends
|
+-- /project/project-b/                 <-- Project B Dashboard
|   +-- /batches
|   +-- /queue
|   +-- /history
|   +-- /analytics
|
+-- /project/project-c/                 <-- Project C Dashboard
|   +-- (same structure)
|
+-- /system/                            <-- Cross-project views
    +-- /daemon                         <-- Daemon health, uptime
    +-- /all-active                     <-- All active work across projects
    +-- /config                         <-- projects.toml, LLM quota
    +-- /quota                          <-- LLM usage (Claude, MiniMax)
```

Every page is project-scoped via URL. The sidebar has a project selector dropdown. Switching projects changes the URL prefix and all data queries filter by `project_id`.

4.3. How the Database Handles Multiple Projects

Every table uses `project_id` as part of its primary key or foreign key:

```
-- Each project has its own ID sequences
-- InnoForge: F001..F141, I001..I195
-- Project B: F001, I001 (fresh start)
-- No collisions because of composite keys

SELECT * FROM id_sequences;
-- project_id | prefix | next_number
-- innoforge  | F      | 1           (fresh start)
-- innoforge  | I      | 1
-- project-b  | F      | 1
-- project-b  | I      | 1
-- project-c  | F      | 1
-- project-c  | I      | 1

-- Work items are globally unique per project
SELECT * FROM work_items WHERE project_id = 'innoforge' AND id = 'I001';
-- Different from project-b's I001

-- Batches are project-scoped
SELECT * FROM batches WHERE project_id = 'innoforge';
-- Only shows InnoForge batches
```

4.4. How the Daemon Manages Multiple Projects

```
class MultiProjectDaemon:
    """Single process managing all registered projects."""

    def main_loop(self):
        while self.running:
            # Reload project config if changed (SIGHUP or periodic)
            self.reload_projects_if_needed()

            for project_id, config in self.projects.items():
                if not config.enabled:
                    continue

                # Each project gets its own processing
                manager = self.managers[project_id]

                # 1. Check for approved batches to start
                # 2. Monitor executing items
                # 3. Process merge queue (one merge at a time per project)
                # 4. Check for stalled items
                # 5. Update git status
                manager.process_cycle()

            # Cross-project services
            self.quota_monitor.poll_if_due()
            time.sleep(60)
```

Key isolation guarantees:

- Each project has its own `BatchManager` instance
- Merge queues are per-project (InnoForge merges don't block Project B)
- Worktrees are in each project's own repo (naturally isolated)
- ID allocation is per-project (no cross-project collisions)
- LLM quota is shared (it's the same Claude account) — the quota monitor tracks this globally

4.5. Cross-Project Data Flow

The split is clean:

- **DB** stores state: IDs, statuses, timestamps, step results, analytics
- **Files** store content: design docs, prompts, reports, code
- **Dashboard** reads DB to render pages, links to files for content display
- **Daemon** reads/writes DB for orchestration, calls executor scripts for file operations
- **Agents** read files for context, call `iw` CLI to update DB state

---

5. Skills Distribution Model

See Diagram 6: Skills Distribution

5.1. The Problem with Current Skills

Currently, skills live inside InnoForge's `.claude/skills/` directory. They can't be reused by other projects without copying. When a skill is updated, each project needs to be manually updated.

5.2. The Package Model

The `iw` CLI includes a skill management system that works like a package manager:

```
iw-ai-core/skills/              <-- Master copies (source of truth)
+-- iw-new-incident/
|   +-- SKILL.md                <-- version: 2.1.0
|   +-- helpers/
+-- iw-new-feature/
|   +-- SKILL.md                <-- version: 2.1.0
+-- iw-batch-execute/
|   +-- SKILL.md                <-- version: 1.5.0
+-- iw-workflow/
|   +-- SKILL.md                <-- version: 3.0.0
+-- ... (all platform skills)
```

5.3. Skill Lifecycle

Installation (one-time per project)

```
iw init-project --id my-project --path /path/to/repo
```

This command:
1. Creates `.iw-orch.json` in the project root
2. Registers the project in `projects.toml`
3. Creates `ai-dev/design/active/`, `ai-dev/design/done/`, `ai-dev/workflow.md`
4. Copies all base skills to `.claude/skills/`
5. Creates `.iw-skills-lock.json` tracking installed versions:

```
{
  "synced_at": "2026-04-07T10:30:00Z",
  "platform_version": "1.0.0",
  "skills": {
    "iw-new-incident": { "version": "2.1.0", "source": "platform", "overridden": false },
    "iw-new-feature": { "version": "2.1.0", "source": "platform", "overridden": false },
    "innoforge-testing": { "version": "1.0.0", "source": "project", "overridden": true }
  }
}
```

Sync (update skills from platform)

```
iw sync-skills --project innoforge
```

This command:
1. Reads the project's `.iw-skills-lock.json`
2. Compares with master versions in iw-ai-core
3. For each outdated skill that is NOT overridden: copies the new version
4. For overridden skills: shows a warning but does NOT overwrite
5. Updates `.iw-skills-lock.json`

```
$ iw sync-skills --project innoforge
Checking skills for innoforge...
  iw-new-incident: 2.0.0 -> 2.1.0 (updated)
  iw-new-feature: 2.0.0 -> 2.1.0 (updated)
  iw-batch-execute: 1.5.0 (up to date)
  innoforge-testing: project override (skipped)
Synced 2 skills. 1 skipped (project override).
```

Project Overrides

A project can override any base skill by creating its own version in `.claude/skills/<skill-name>/`. When `iw sync-skills` runs, it detects the override and marks it in the lock file:

```
# Project creates its own version of iw-new-incident
mkdir .claude/skills/iw-new-incident/
# Write custom SKILL.md with project-specific logic

# Next sync detects the override
iw sync-skills --project innoforge
# Output: iw-new-incident: project override (skipped)
```

To revert to the platform version:

```
iw sync-skills --project innoforge --force iw-new-incident
```

5.4. How Skills Reference the `iw` CLI

Every skill that needs to interact with the database uses the `iw` CLI. The skill's `SKILL.md` contains instructions like:

```
## Step 1: Reserve Incident ID

Run this command to get the next available incident ID:

\`\`\`bash
iw next-id --type incident --project $(iw current-project)
\`\`\`

The `iw current-project` command reads `.iw-orch.json` from the repo root
to determine which project this is.
```

The `iw` CLI is installed once on the machine (`pip install -e /path/to/iw-ai-core`) and available on PATH. It works from any directory — it finds the project by looking for `.iw-orch.json` up the directory tree.

---

6. Database Architecture

See Diagram 7: Database Schema (ER Diagram)

6.1. Design Principles

- **DB is the single source of truth for ALL operational state**
- Files store content only (design docs, prompts, reports)
- Every table has `project_id` for multi-project isolation
- Composite primary keys: `(project_id, id)` for work items and batches
- Atomic ID allocation via `FOR UPDATE` row locks
- All timestamps in UTC with timezone

6.2. Core Tables

```
projects
 |-- id (PK): "innoforge", "project-b"
 |-- display_name, repo_root, config (JSONB), enabled
 |
 +-- id_sequences (project_id, prefix -> next_number)
 |   Atomic counter: F->1, I->1, CR->1, BATCH->1 (fresh start per project)
 |
 +-- work_items (project_id, id -> type, title, status, phase, ...)
 |   |   Status: draft -> approved -> in_progress -> completed | failed
 |   |   Phase: active -> work -> done
 |   |
 |   |   Tier 1 content (always viewable in dashboard):
 |   |   - design_doc_content TEXT       (full markdown of design doc)
 |   |   - design_doc_search  TSVECTOR   (PostgreSQL full-text search index)
 |   |   - summary            TEXT       (AI-generated 2-3 line summary)
 |   |
 |   |   Tier 2 archive (on-demand extraction):
 |   |   - archive_path       TEXT       (path to .tar.zst in archive/)
 |   |   - archive_size_bytes BIGINT
 |   |   - archived_at        TIMESTAMPTZ
 |   |
 |   |   Future RAG (pgvector):
 |   |   - design_doc_embedding VECTOR(1536)
 |   |
 |   +-- workflow_steps (project_id, work_item_id -> step_number, agent, status, ...)
 |       |   Status: pending -> in_progress -> completed | failed | needs_fix
 |       |   - report_content TEXT  (full report markdown, Tier 1)
 |       |
 |       +-- step_runs (step_id -> THE EXECUTION CONTROL PLANE)
 |       |   Each launch/retry creates a new row. Tracks everything for restart.
 |       |   - run_number, status (pending/running/completed/failed/timeout/killed/stalled)
 |       |   - pid INTEGER         (OS process ID — enables kill, alive-check)
 |       |   - pid_alive BOOLEAN   (daemon sets via kill -0 each poll cycle)
 |       |   - command TEXT        (exact launch command — enables one-click restart)
 |       |   - worktree_path TEXT  (where the agent runs)
 |       |   - cli_tool TEXT       ("opencode" | "claude")
 |       |   - last_heartbeat TIMESTAMPTZ  (stall detection)
 |       |   - timeout_secs INTEGER (dynamic per step type, not global 30-min)
 |       |   - error_message TEXT  (human-readable failure reason)
 |       |   - exit_code, log_file, report_file, started_at, completed_at, duration_secs
 |       |
 |       +-- fix_cycles (step_id -> cycle_number, trigger_type, status, ...)
 |
 +-- batches (project_id, id -> status, max_parallel, ...)
 |   |   Status: planning -> approved -> executing -> completed | completed_with_errors
 |   |
 |   +-- batch_items (project_id, batch_id, work_item_id -> group, status, pid, ...)
 |       Status: pending -> setting_up -> executing -> completed -> merged | failed
 |
 +-- migration_locks (project_id -> current_holder, branch, locked_at)
 |   Only one item per project can create Alembic migrations
```

```
|
+-- daemon_events (project_id, event_type, entity_id, message, metadata, created_at)
    Audit trail for analytics and dashboard notifications
```

6.3. ID Allocation — The Fix for Race Conditions

Before (markdown files — broken under concurrency):

```
Agent A reads Incident_tracking.md -> sees I195
Agent B reads Incident_tracking.md -> sees I195  (RACE!)
Agent A appends I196 -> file now shows I196
Agent B appends I196 -> DUPLICATE ID!
```

After (PostgreSQL with row locks — bulletproof):

```
Agent A: iw next-id --type incident --> DB locks row, returns I001, increments to I002
Agent B: iw next-id --type incident --> waits for lock, gets I002, increments to I003
        (B waited ~1ms for A's transaction to commit)
```

No race conditions. No duplicates. No stale reads. The `FOR UPDATE` lock serializes concurrent access at the row level.

---

7. Two-Tier Content Storage Architecture

See Diagram 4: Data Flow Architecture (updated with two-tier model)

7.1. The Problem: Repository Bloat

Each completed work item generates ~10-20 files (design doc, 4-8 prompts, 4-8 reports, evidences). At production velocity:
- ~370+ completed items over InnoForge's lifetime
- Each item averages ~50-100 KB of text + binary evidences
- This accumulates in `ai-dev/design/done/`, bloating git history permanently
- Git never forgets — even deleted files remain in the object store

This content has **zero runtime value** to the project. It's historical audit trail for the AI workflow.

7.2. The Solution: Tier 1 (DB) + Tier 2 (Archive)

TIER	CONTENT	STORAGE	ACCESS	SEARCH
Tier 1: Always hot	Design docs, reports, summaries, step outcomes	PostgreSQL `TEXT` columns	Instant (DB query)	Full-text search + future RAG
Tier 2: On-demand	Prompts, evidences, screenshots, logs, raw artifacts	`.tar.zst` in `iw-ai-core/archive/`	Extract to tmp on request	By metadata only (DB)

Active work (in-flight items being designed or executed) stays in the project repo at `ai-dev/design/active/`. Agents need filesystem access during execution. Once an item is completed and merged, `iw archive` migrates it to the two-tier system and removes it from the project repo.

7.3. Tier 1: Searchable Content in PostgreSQL

Design documents and reports are stored as `TEXT` columns directly in the database. A typical design doc is 5-15 KB of markdown — trivial for PostgreSQL.

```
-- Work item content (always viewable in dashboard)
work_items.design_doc_content  TEXT      -- Full markdown of the design document
work_items.design_doc_search   TSVECTOR  -- PostgreSQL full-text search index
work_items.summary             TEXT      -- AI-generated 2-3 line summary for list views

-- Step report content (always viewable in dashboard)
workflow_steps.report_content  TEXT      -- Full report markdown per step

-- Full-text search index
CREATE INDEX idx_work_items_fts ON work_items USING GIN(design_doc_search);
```

What this enables:
- Click on any work item in the dashboard — design doc renders instantly, no file I/O
- Full-text search: “find all incidents mentioning template rendering” across all projects
- List views show AI summaries, not just titles
- Reports for every step viewable inline — see exactly what each agent did

7.4. Tier 2: Compressed Archive for Full Artifacts

Everything beyond the design doc and reports is compressed into a single archive per work item:

```
iw-ai-core/
  archive/                    <-- NOT in git (.gitignore)
    innoforge/
      I001.tar.zst            <-- ~20-50 KB compressed
      I002.tar.zst
      F089.tar.zst
    project-b/
      I001.tar.zst
```

Archive format: `.tar.zst` (Zstandard compression — 3-5x faster decompression than gzip, better ratio). A typical work item compresses from ~200 KB to ~40 KB.

The archive directory is NOT tracked in git. It’s operational storage:
- Backed up separately (rsync, S3, etc.)
- The DB has full metadata for every archived item
- If an archive file is lost, the DB still has the design doc and reports (Tier 1)

```
-- Archive metadata in DB
work_items.archive_path       TEXT          -- 'innoforge/I001.tar.zst'
work_items.archive_size_bytes  BIGINT       -- compressed size
work_items.archived_at        TIMESTAMPTZ   -- when it was archived
```

7.5. On-Demand Artifact Viewing

When a user wants to see the full artifacts for a completed item:

```
User clicks "View Full Artifacts" on I001 in dashboard
  |
  v
Dashboard: GET /project/innoforge/item/I001/artifacts
  |
  v
Backend checks: is there an active extraction in tmp?
  |
  +-- No:
  |   1. Read archive_path from DB: "innoforge/I001.tar.zst"
  |   2. Extract to: /tmp/iw-archive-view/innoforge/I001/
  |   3. Set TTL timer (cleanup after 10 min of inactivity)
  |   4. Return file tree listing
  |
  +-- Yes:
  |   1. Reuse existing extraction
  |   2. Reset TTL timer
  |
  v
Dashboard renders file browser:
  - Prompts (markdown, rendered inline)
  - Evidences (images, displayed inline)
  - Reports (already in DB, but raw files also available)
  - Logs (plain text, scrollable)
  - workflow-manifest.json (original snapshot)
  |
  v
User navigates away -> TTL expires -> rm -rf /tmp/iw-archive-view/innoforge/I001/
```

Performance: Extracting a 40 KB `.tar.zst` takes <50ms. The user won't notice any delay.

Everything extracted is read-only. The archive is the source of truth. No editing through the dashboard.

7.6. The `iw archive` Command — Complete Flow

```
iw archive I001 --project innoforge
```

Step 1 — Tier 1: Store searchable content in DB

```
# Read design doc from project repo
design_path = project.ai_dev / "design" / "active" / "I001" / "I001_Issue_Design.md"
design_content = design_path.read_text()

# Store in DB
db.execute("""
    UPDATE work_items
    SET design_doc_content = :content,
        design_doc_search = to_tsvector('english', :content),
        phase = 'done',
        archived_at = now()
    WHERE project_id = :project AND id = :item_id
""", content=design_content, project='innoforge', item_id='I001')

# Store each step report
for step in steps:
    report_path = work_dir / "reports" / step.report_file
    if report_path.exists():
        db.execute("""
            UPDATE workflow_steps SET report_content = :content
            WHERE id = :step_id
        """, content=report_path.read_text(), step_id=step.id)
```

Step 2 — Tier 2: Compress to archive

```
# Compress entire work item folder
tar -cf - -C ai-dev/design/active I001 | zstd -3 -o archive/innoforge/I001.tar.zst

# Record in DB
UPDATE work_items SET
  archive_path = 'innoforge/I001.tar.zst',
  archive_size_bytes = <file_size>
WHERE project_id = 'innoforge' AND id = 'I001';
```

Step 3 — Clean up project repo

```
# Remove from project repo (it's all in DB + archive now)
rm -rf ai-dev/design/active/I001/
```

Optional Step 4 — Generate AI summary

```
# Call LLM for a concise summary (useful for list views, search results)
summary = llm.summarize(design_content, max_tokens=100)
db.execute("UPDATE work_items SET summary = :s WHERE ...", s=summary)
```

7.7. Full-Text Search

PostgreSQL's built-in `tsvector` provides fast, accurate full-text search at zero infrastructure cost:

```
-- Search across all projects
SELECT project_id, id, title, summary,
       ts_rank(design_doc_search, query) AS relevance
FROM work_items, to_tsquery('english', 'template & rendering & timeout') AS query
WHERE design_doc_search @@ query
ORDER BY relevance DESC
LIMIT 20;

-- Search within a single project
SELECT id, title, summary
FROM work_items
WHERE project_id = 'innoforge'
  AND design_doc_search @@ to_tsquery('english', 'WeasyPrint & PDF')
ORDER BY ts_rank(design_doc_search, to_tsquery('english', 'WeasyPrint & PDF')) DESC;
```

Dashboard search bar: Type a query, see results across all projects with relevance ranking. Each result shows the title, AI summary, project, type, and completion date.

7.8. Future: RAG with pgvector

When semantic search is needed (e.g., “find incidents similar to this bug description”):

```
-- Add vector column (requires pgvector extension)
ALTER TABLE work_items ADD COLUMN design_doc_embedding vector(1536);
CREATE INDEX idx_work_items_embedding ON work_items USING ivfflat(design_doc_embedding);

-- Similarity search
SELECT id, title, summary, 1 - (design_doc_embedding <=> :query_embedding) AS similarity
FROM work_items
WHERE project_id = 'innoforge'
ORDER BY design_doc_embedding <=> :query_embedding
LIMIT 10;
```

Implementation path:

1. **Day 1 (MVP)**: `tsvector` full-text search (PostgreSQL built-in, zero setup)
2. **Day 2**: pgvector extension + embedding generation on archive
3. **Day 3**: RAG pipeline — embed user query, find similar items, use as context for LLM responses

The content is already in the DB (Tier 1), so adding embeddings is just an additional column + a batch job that runs `iw embed --project innoforge --all`.

7.9. Dashboard Views for Archived Content

```
/project/innoforge/item/I001
  |
  +-- Overview tab (always instant, from DB)
  |   - Title, status, type, dates
  |   - AI summary (2-3 lines)
  |   - Step pipeline with outcomes (pass/fail, fix cycles, durations)
  |   - Key metrics
  |
  +-- Design Document tab (always instant, from DB)
  |   - Full design doc rendered as HTML from work_items.design_doc_content
  |   - Searchable, linkable, always available
  |
  +-- Reports tab (always instant, from DB)
  |   - Each step's report rendered from workflow_steps.report_content
  |   - Review findings, fix cycle history
  |   - Color-coded by severity
  |
  +-- Full Artifacts tab (on-demand, from archive)
      - Click "Load Artifacts" button
      - Backend extracts archive to tmp
      - File tree browser appears:
        - Prompts (markdown, rendered)
        - Evidences (images, inline)
        - Logs (plain text, scrollable)
        - Original manifest snapshot
      - Auto-cleanup on navigate away (TTL-based)
```

7.10. What This Means for Project Repos

Before (current system):

```
ai-dev/
  design/
    active/I194/    <-- in-flight
    active/I195/    <-- in-flight
    done/I001/      <-- 195 completed items accumulating forever
    done/I002/
    ...
    done/I193/      <-- git history bloated with 370+ folders
  tracking/         <-- markdown files with race conditions
  work/             <-- JSON manifests scattered everywhere
```

After (iw-ai-core):

```
ai-dev/
  design/
    active/I001/    <-- only in-flight items, typically 0-10
    active/I002/
  workflow.md       <-- workflow definition
```

The project repo only ever has a handful of active work items. Everything completed lives in the platform's DB (searchable) and archive (on-demand). Git stays lean.

---

8. Component Architecture

8.1. The `iw` CLI

The bridge between LLM agents and the database. Installed via pip, available on PATH.

Command groups:

GROUP	COMMANDS	USED BY
ID management	`iw next-id`, `iw current-project`	Skills (via Claude)
Work items	`iw register`, `iw approve`, `iw archive`	Skills, Human
Steps	`iw step-start`, `iw step-done`, `iw step-fail`	Agents (via orchestrator skill)
Batches	`iw batch-create`, `iw batch-approve`, `iw batch-status`	Skills, Human
Migration lock	`iw migration-lock acquire/release/status`	Agents
Skills	`iw sync-skills`, `iw init-project`	Human
System	`iw daemon start/stop/status`, `iw projects list`	Human

8.2. The Daemon

Single Python process, single-threaded polling loop. Manages all projects.

Responsibilities:
- Poll DB for approved/executing batches across all projects
- Launch worktree setup + agent for each item
- Monitor running steps (PID tracking, heartbeat, stall detection)
- Process merge queue (one merge at a time per project)
- Check auto-publish after batch completion
- Monitor LLM quota
- Track git status per project
- Emit events to `daemon_events` table

8.3. The Executor Scripts

Deterministic bash scripts. No LLM involvement. Project-agnostic.

SCRIPT	PURPOSE
worktree_setup.sh	Create worktree, install deps, write execution_brief.json, sync skills
step_executor.sh	Launch LLM agent for a step, handle timeouts, parse results
worktree_commit.sh	Squash-merge worktree branch to project's main
batch_dispatcher.sh	Legacy compatibility wrapper (daemon now handles this directly)

8.4. The Dashboard

FastAPI + Jinja2 + htmx + SSE. No React SPA. No build pipeline.

Key features:

- Project selector with status badges
- Batch overview with execution timeline (Gantt)
- Work item detail with step pipeline visualization
- **Running Tasks view** — all active steps across all projects with live durations
- One-click recovery actions (restart, kill, skip — all DB-backed)
- SSE-powered toast notifications
- LLM quota footer
- Analytics (success rates, fix cycles, agent effectiveness)
- Git status panel per project

9. Execution Management — DB-Backed Process Control

This section describes how the platform manages step execution end-to-end using the database as the single source of truth. **No file editing, no manifest tracing, no manual PID hunting.** Everything is queryable, actionable, and visible from the dashboard.

9.1. The Problem with File-Based Execution Management

In the current InnoForge system, managing running tasks requires:

TASK	WHAT YOU HAVE TO DO TODAY
Check what's running	`ps aux | grep opencode`, inspect worktrees, read manifest JSONs
See how long a step has been running	Check file timestamps in `ai-dev/work/`
Restart a failed step	Find the manifest JSON, edit `status` fields, figure out which worktree, manually re-launch the agent
Kill a hanging task	Find the PID by reading process tables, `kill` from terminal
Detect a stalled process	Manually check if a process is alive and whether it's making progress
Restart from step N	Edit multiple manifest entries, reset statuses for all downstream steps, re-launch

This is error-prone, slow, and requires deep knowledge of the file structure. The entire execution model must be DB-backed.

9.2. Step Runs — The Execution Control Plane

The `step_runs` table is the operational control plane for all agent execution. Every time a step launches (or re-launches), a new row is created:

```
CREATE TABLE step_runs (
    id              SERIAL PRIMARY KEY,
    step_id         INTEGER NOT NULL REFERENCES workflow_steps(id),
    run_number      INTEGER NOT NULL,              -- 1, 2, 3 (each retry = new run)
    status          TEXT NOT NULL DEFAULT 'pending',-- pending/running/completed/failed/timeout/killed/stalled
    pid             INTEGER,                       -- OS process ID of the LLM session
    pid_alive       BOOLEAN DEFAULT false,         -- daemon sets this on each poll cycle
    command         TEXT,                          -- exact launch command (for restart)
    worktree_path   TEXT,                          -- where the agent is running
    cli_tool        TEXT,                          -- "opencode" | "claude"
    exit_code       INTEGER,
    log_file        TEXT,                          -- path to step log
    report_file     TEXT,                          -- path to report (also in DB Tier 1)
    started_at      TIMESTAMPTZ,
    completed_at    TIMESTAMPTZ,
    duration_secs   FLOAT,
    last_heartbeat  TIMESTAMPTZ,                   -- updated by daemon if PID alive
    timeout_secs    INTEGER,                       -- dynamic timeout per step type
    error_message   TEXT                           -- human-readable failure reason
);
```

Key fields explained:

FIELD	PURPOSE	WHO WRITES
`pid`	OS process ID — enables kill, alive-check	Daemon (on launch)
`pid_alive`	Is the process still running right now?	Daemon (every poll cycle via `kill -0`)
`command`	Exact shell command used to launch	Daemon (on launch) — enables one-click restart
`worktree_path`	Full path to the worktree where agent runs	Daemon (from worktree_setup)
`last_heartbeat`	Last time daemon confirmed PID was alive	Daemon (every poll cycle)
`timeout_secs`	How long this step is allowed to run	Set per step type (dynamic, not a global constant)
`error_message`	Why it failed — timeout, crash, review findings	Daemon or agent (via `iw step-fail --reason "..."`)

9.3. Dynamic Timeouts Per Step Type

The current system uses a global 30-minute timeout that kills complex tasks mid-work. The new system uses per-step-type timeouts:

STEP TYPE	DEFAULT TIMEOUT	RATIONALE
Implementation (Backend, Frontend, API, DB)	45 min	Large features need more time
Code Review (per-agent)	30 min	Reviews are focused
Code Review (global/final)	40 min	Reviews multiple agents' work
Code Review Fix	45 min	Fix agent needs to read review + fix + retest
Quality Validation (lint, format)	10 min	Fast, script-driven
Quality Validation (tests)	20 min	Integration tests can be slow
Quality Validation (browser)	15 min	Playwright verification
QV Fix (LLM-driven)	30 min	Auto-fix lint/type/test failures

Timeouts are configurable per project in `.iw-orch.json`:

```
{
  "timeout_overrides": {
    "implementation": 3600,
    "code_review": 2400,
    "quality_validation_tests": 1800
  }
}
```

9.4. Daemon Poll Cycle — Step Monitoring

Every 60 seconds, the daemon checks all running steps across all projects:

```
def monitor_running_steps(self):
    """Check health of all running step_runs across all projects."""
    running = db.query(StepRun).filter(StepRun.status == 'running').all()

    for run in running:
        now = datetime.utcnow()

        # 1. Check if process is still alive
        try:
            os.kill(run.pid, 0)  # signal 0 = existence check, no actual signal sent
            alive = True
        except (ProcessLookupError, PermissionError):
            alive = False

        run.pid_alive = alive

        if alive:
            run.last_heartbeat = now

            # 2. Check timeout
            elapsed = (now - run.started_at).total_seconds()
            if elapsed > run.timeout_secs:
                os.kill(run.pid, signal.SIGTERM)
                run.status = 'timeout'
                run.error_message = f'Exceeded {run.timeout_secs}s timeout after {elapsed:.0f}s'
                run.completed_at = now
                run.duration_secs = elapsed
                self.emit_event('step_timeout', run)

            # 3. Check stall (alive but no iw CLI calls for 10+ minutes)
            elif run.last_heartbeat and (now - run.last_heartbeat).total_seconds() > 600:
                run.status = 'stalled'
                run.error_message = 'Process alive but no progress for 10+ minutes'
                self.emit_event('step_stalled', run)

        else:
            # 4. Process died without reporting via iw CLI
            if run.status == 'running':
                run.status = 'failed'
                run.error_message = 'Process exited unexpectedly (no completion reported)'
                run.completed_at = now
                run.duration_secs = (now - run.started_at).total_seconds()
                self.emit_event('step_crashed', run)

        db.commit()
```

9.5. Dashboard: Running Tasks View

A single page showing all active execution across all projects:

```
/system/running

+=============================================================================+
| RUNNING NOW                                    3 steps active   |
+=============================================================================+
| Project   | Item | Step          | Agent     | PID   | Duration | Actions |
|-----------|------|---------------|-----------|-------|----------|---------|
| InnoForge | I003 | S01 Backend   | back-impl | 45231 | 12m 34s  | [Kill]  |
| InnoForge | I004 | S03 CR Final  | cr-final  | 45298 |  4m 12s  | [Kill]  |
| Project B | F002 | S02 API       | api-impl  | 45301 |  1m 05s  | [Kill]  |
+=============================================================================+

+=============================================================================+
| FAILED / NEEDS ATTENTION                          2 items        |
+=============================================================================+
| Project   | Item | Step          | Reason           | Duration | Actions  |
|-----------|------|---------------|------------------|----------|----------|
| InnoForge | I001 | S02 CR Backend| Timeout (30m)    | 30m 00s  |[Restart] |
|           |      |               |                  |          |[Skip]    |
| Project B | F001 | S05 QV Tests  | Tests failed (3) |  5m 12s  |[Restart] |
|           |      |               |                  |          |[Skip]    |
+=============================================================================+

+=============================================================================+
| RECENTLY COMPLETED (last hour)                    8 steps        |
+=============================================================================+
| Project   | Item | Step          | Agent     | Duration | Result          |
|-----------|------|---------------|-----------|----------|-----------------|
| InnoForge | I003 | S00 Setup     | (bash)    |  2m 15s  | OK              |
| InnoForge | I002 | S04 QV Lint   | (bash)    |  0m 45s  | OK              |
| InnoForge | I002 | S05 QV Types  | (bash)    |  1m 22s  | OK              |
| InnoForge | I002 | S06 QV Tests  | (bash)    |  3m 10s  | OK (run 2)      |
| Project B | F002 | S01 Database  | db-impl   | 18m 22s  | OK              |
+=============================================================================+
```

Live updates via SSE: Duration counters update in real-time. New entries appear/disappear as steps start/finish. No page refresh needed.

Per-project filtered view also available at `/project/innoforge/running` — same layout, filtered to one project.

9.6. One-Click Actions — How They Work

All actions are DB mutations. The daemon picks up changes on its next poll cycle (within 60 seconds). For kill actions, the daemon sends the signal immediately.

Kill a Running Step

```
User clicks [Kill] on I003/S01

Dashboard: POST /project/innoforge/api/item/I003/kill-step/S01

Backend:
  1. Find active step_run for I003/S01 where status='running'
  2. os.kill(step_run.pid, signal.SIGTERM)
  3. UPDATE step_runs SET status='killed', completed_at=now(),
     error_message='Killed by user from dashboard'
  4. UPDATE workflow_steps SET status='failed'
  5. Emit daemon_event: 'step_killed'
  6. Return 200 OK

Dashboard: Toast "I003/S01 killed. [Restart] [Skip]"
```

Restart a Failed Step

```
User clicks [Restart] on I001/S02

Dashboard: POST /project/innoforge/api/item/I001/restart-step/S02

Backend:
  1. Find last step_run for I001/S02
  2. Verify step is restartable (failed | timeout | killed | stalled)
  3. INSERT INTO step_runs (
       step_id, run_number, status, command, worktree_path,
       cli_tool, timeout_secs
     ) VALUES (
       :step_id, :prev_run_number + 1, 'pending',
       :same_command,    -- reuse exact command from failed run
       :same_worktree,   -- reuse same worktree (still exists)
       :same_cli_tool, :same_timeout
     )
  4. UPDATE workflow_steps SET status='pending'
  5. UPDATE work_items SET status='in_progress' (if it was 'failed')
  6. Return 200 OK

Daemon (next poll, <=60s):
  1. Finds step_run with status='pending'
  2. cd <worktree_path> && <command>
  3. UPDATE step_runs SET status='running', pid=<new_pid>,
     started_at=now(), pid_alive=true
```

The key insight: `step_runs.command` stores the exact launch command (e.g., `opencode run '/execute I001 S02'`). Restart doesn't need to figure anything out — it just re-runs the stored command in the stored worktree.

Skip a Step

```
User clicks [Skip] on I001/S02

Dashboard: POST /project/innoforge/api/item/I001/skip-step/S02

Backend:
  1. UPDATE workflow_steps SET status='completed' (manual override)
  2. INSERT INTO step_runs (step_id, run_number, status, error_message)
     VALUES (:step_id, :next, 'completed', 'Skipped by user from dashboard')
  3. Daemon advances to next step on next poll
```

Restart from Step N

```
User clicks "Restart from S03" on I001

Dashboard: POST /project/innoforge/api/item/I001/restart-from/S03

Backend:
  1. For all workflow_steps where step_number >= 3:
     - UPDATE workflow_steps SET status='pending',
       started_at=NULL, completed_at=NULL
  2. Create new step_run with status='pending' for S03
  3. UPDATE work_items SET status='in_progress'
  4. Daemon picks up S03 on next poll, then continues S04, S05... sequentially
```

9.7. Step Launch Flow — What the Daemon Records

When the daemon launches a new step, it records everything needed for future restart:

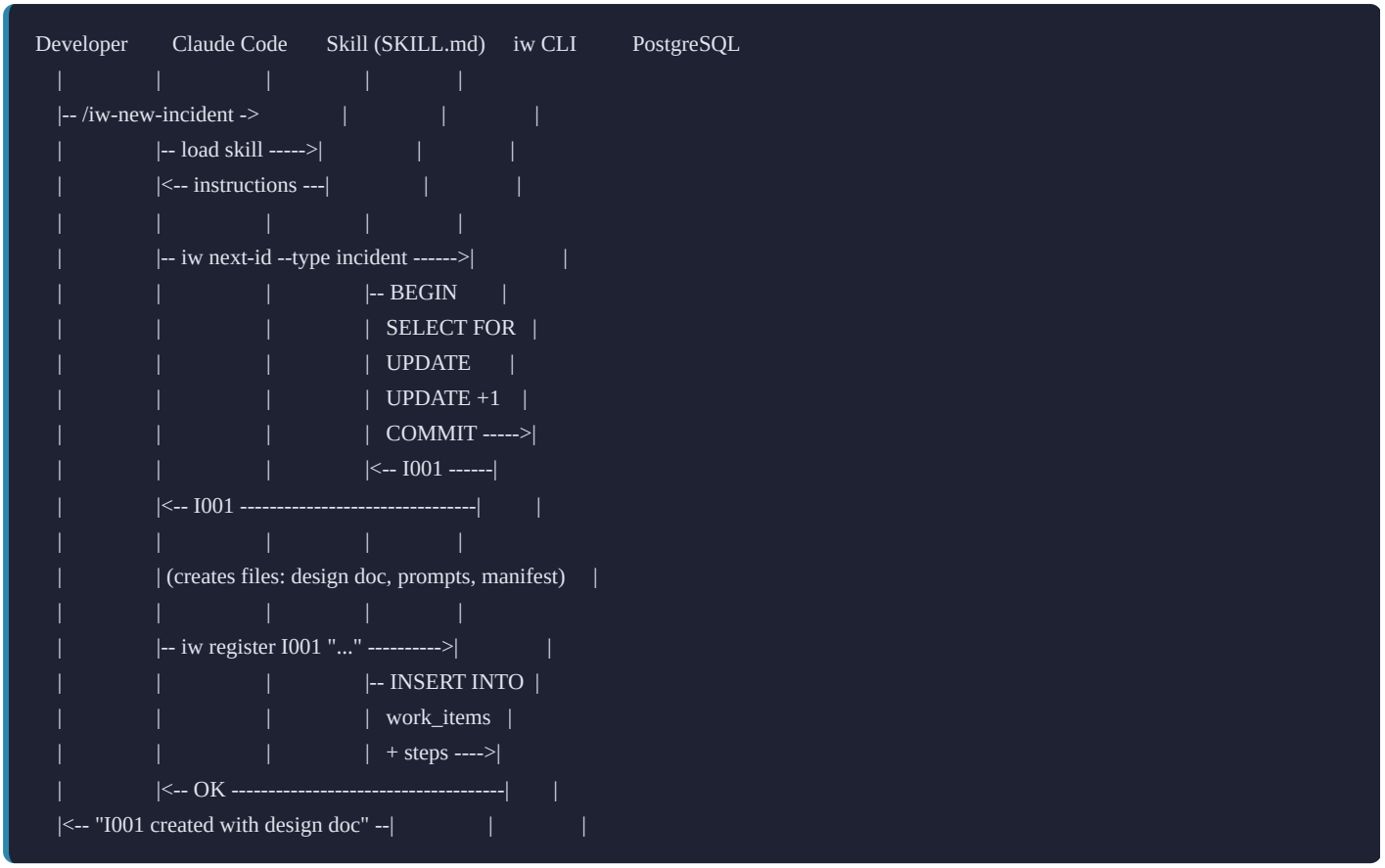
```
def launch_step(self, project, item, step, worktree_path):
    """Launch an agent for a workflow step. Record everything for restart."""

    # Build the exact command
    cli_tool = project.config.get('cli_tool', 'opencode')
    if cli_tool == 'opencode':
        command = f"opencode run '/execute {item.id} {step.step_id}'"
    else:
        command = f"claude -p '/execute {item.id} {step.step_id}'"

    # Determine timeout
    timeout = self.get_timeout(project, step.step_type)

    # Launch the process
    proc = subprocess.Popen(
        command, shell=True, cwd=worktree_path,
        stdout=open(log_file, 'w'), stderr=subprocess.STDOUT
    )

    # Record everything in DB
    run = StepRun(
        step_id=step.id,
        run_number=step.current_run + 1,
        status='running',
        pid=proc.pid,
        pid_alive=True,
        command=command,
        worktree_path=str(worktree_path),
        cli_tool=cli_tool,
        log_file=str(log_file),
        started_at=datetime.utcnow(),
        last_heartbeat=datetime.utcnow(),
        timeout_secs=timeout,
    )
    db.add(run)

    step.status = 'in_progress'
    step.started_at = datetime.utcnow()
    db.commit()

    self.emit_event('step_launched', item_id=item.id, step_id=step.step_id, pid=proc.pid)
```

9.8. Summary: What the DB Eliminates

PAIN POINT (FILE-BASED)	DB SOLUTION
“What’s running?” — grep PIDs, inspect worktrees	`SELECT * FROM step_runs WHERE status='running'`
“How long has it been running?” — check timestamps	`started_at` column — dashboard shows live counter
“Restart a failed step” — edit manifests, find worktree, re-launch	Click [Restart] — DB has command + worktree, daemon re-launches
“Is this process stuck?” — `ps aux`, read logs	`pid_alive` + `last_heartbeat` — automatic stall detection
“Kill a hanging task” — find PID, terminal kill	Click [Kill] — SIGTERM via API, immediate
“What went wrong?” — dig through log files	`error_message` column — one glance in dashboard
“Restart from step 3” — edit N manifest entries	Click “Restart from S03” — resets all downstream, daemon continues
“30-min timeout kills everything” — global constant	Dynamic timeouts per step type, configurable per project
“Zombie processes” — manual cleanup	Daemon detects dead PIDs, auto-marks as failed

10. Detailed Interaction Diagrams

9.1. Skill-to-Database Flow (ID Allocation)



9.2. Agent Execution Flow (Inside Worktree)

9.3. Dashboard Data Flow

```
Browser          Dashboard (FastAPI)          PostgreSQL
   |                    |                         |
   |-- GET /project/innoforge/batches ->          |
   |                    |-- SELECT * FROM batches ->|
   |                    |   WHERE project_id =     |
   |                    |   'innoforge'            |
   |                    |<-- rows -------------------|
   |                    |                         |
   |                    |-- Render Jinja2 template |
   |<-- HTML page ------|                          |
   |                    |                         |
   |-- SSE: /api/stream/notifications ->          |
   |                    |-- SELECT FROM            |
   |                    |   daemon_events          |
   |                    |   WHERE created_at >     |
   |                    |   last_check ----------->|
   |                    |<-- new events -------------|
   |<-- SSE event ------|                          |
   |   (toast shown)    |                         |
```

11. Migration Plan

9.1. Approach: Clean Cut

Based on the user's decision: **full migration, fresh start, all at once.**

No incremental dual-write bridge. No history import. Shutdown the old system, stand up the new one.

9.2. Migration Steps

Step 1: Create iw-ai-core Repository

```
mkdir /home/sergiog/dev/iw-ai-core
cd /home/sergiog/dev/iw-ai-core
git init
```

Create the directory structure:

```
iw-ai-core/
+-- orch/                    # Python daemon + core logic
|   +-- daemon.py
|   +-- batch_manager.py
|   +-- state_machine.py
|   +-- project_registry.py
|   +-- quota_monitor.py
|   +-- git_status.py
|   +-- cli.py               # iw CLI entry point
|   +-- db/
|       +-- models.py
|       +-- session.py
|       +-- migrations/      # Alembic
+-- executor/                # Bash scripts
|   +-- step_executor.sh
|   +-- step_executor_lib.sh
|   +-- worktree_setup.sh
|   +-- worktree_commit.sh
+-- dashboard/               # FastAPI web dashboard
|   +-- app.py
|   +-- routers/
|   +-- templates/
|   +-- static/
+-- skills/                  # Master skill copies
|   +-- iw-new-incident/
|   +-- iw-new-feature/
|   +-- iw-new-cr/
|   +-- iw-batch-execute/
|   +-- iw-workflow/
|   +-- ... (all platform skills)
+-- templates/               # Default workflow templates
|   +-- Feature_Design_Template.md
|   +-- Issue_Design_Template.md
|   +-- ...
+-- projects.toml            # Project registry
+-- docker-compose.yml       # PostgreSQL (port 5433)
+-- pyproject.toml
+-- Makefile
+-- CLAUDE.md
```

Step 2: Set Up PostgreSQL

```
# docker-compose.yml
services:
  db:
    image: postgres:15
    ports:
      - "5433:5432"
    environment:
      POSTGRES_DB: iw_orch
      POSTGRES_USER: iw_orch
      POSTGRES_PASSWORD: iw_orch_dev
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

Run Alembic migrations to create all tables.

Step 3: Port Existing Code

FROM (INNOFORGE)	TO (IW-AI-CORE)	CHANGES
`scripts/ai_dev_daemon/`	`orch/`	Add `project_id` parameter to all functions
`scripts/ai_dashboard/`	`dashboard/`	Convert to FastAPI, add project selector, DB queries
`scripts/step_executor.sh`	`executor/step_executor.sh`	Add `project_repo_root` parameter
`scripts/worktree_setup.sh`	`executor/worktree_setup.sh`	Add skill sync, execution_brief generation
`.claude/skills/iw-*`	`skills/iw-*`	Update to use `iw` CLI instead of file-based tracking
`ai-dev/templates/`	`templates/`	Move default templates

Step 4: Register InnoForge

1. Create `.iw-orch.json` in InnoForge's repo root
2. Add entry to `projects.toml`
3. Initialize ID sequences in DB (all start at 1 — fresh start)
4. Run `iw sync-skills --project innoforge`

Step 5: Clean Up InnoForge

Delete from InnoForge:

- `scripts/ai_dev_daemon/` (moved to iw-ai-core)
- `scripts/ai_dashboard/` (moved to iw-ai-core)
- `scripts/step_executor.sh`, `step_executor_lib.sh` (moved)
- `scripts/worktree_setup.sh`, `worktree_commit.sh` (moved)

- `scripts/batch_dispatcher.sh`, `batch_launch.sh` (moved)
- `ai-dev/tracking/` (replaced by DB)
- `ai-dev/work/` (execution state now in DB)
- Makefile targets that reference moved scripts (update to call `iw` CLI)

Keep in InnoForge:

- `ai-dev/design/active/` (only in-flight work items, no `done/` folder — archived content lives in DB + archive)
- `ai-dev/workflow.md` (project-specific workflow definition)
- `ai-dev/templates/` (project-specific prompt templates, if any overrides)
- `.claude/skills/` (synced from iw-ai-core + project-specific overrides)
- `CLAUDE.md` (updated to reference iw-ai-core)
- `Makefile` (updated: `make batch-launch` now calls `iw batch-launch`)

Step 6: Verify & Go Live

1. Start iw-ai-core: `docker compose up -d` (PostgreSQL)
2. Start daemon: `iw daemon start`
3. Open dashboard: `http://localhost:9900`
4. Create a test incident: `/iw-new-incident` in InnoForge
5. Verify ID appears in dashboard
6. Execute the incident
7. Verify end-to-end flow works

---

12. Security & Isolation

10.1. Project Isolation

- **Database**: All queries filter by `project_id`. No cross-project data leaks.
- **File system**: Each project's files stay in its own repo. No cross-repo file access.
- **Worktrees**: Each project's worktrees are under its own `.worktrees/` directory.
- **Merge queues**: Per-project. InnoForge merges don't block other projects.

10.2. Agent Isolation

- Agents run in git worktrees (full repo checkout, isolated branch)
- Agents communicate with DB only via `iw` CLI (no direct DB access)
- Agents never write state files — CLI enforces valid state transitions
- Each agent session is short-lived (one step at a time)

10.3. Dashboard Security

- v1: No authentication (single user, localhost only)
- Future: Token-based auth via environment variable

13. Summary of Data Flow

Where data lives at each stage:

STAGE	STATE (DB)	CONTENT	LOCATION
Design created	work_items: status=draft	Design doc + prompts	Project repo: `ai-dev/design/active/I001/`
Approved	work_items: status=approved	Unchanged	Project repo (same)
Batch planned	batches + batch_items created	No new content	DB only
Worktree created	batch_items: status=setting_up	execution_brief.json	Worktree (`.worktrees/I001/`)
Agent executing	workflow_steps: in_progress	Code changes + reports	Worktree
Step completed	workflow_steps: completed	Report file written	Worktree
All steps done	work_items: completed	All reports exist	Worktree
Merged	batch_items: merged	Code on main branch	Main branch
Archived	work_items: phase=done	**Tier 1**: design doc + reports in DB	PostgreSQL (always viewable)
	archived_at, archive_path set	**Tier 2**: everything else compressed	`archive/innoforge/I001.tar.zst`
		Project repo cleaned	`ai-dev/design/active/I001/` deleted

Appendix A: Comparison — Before and After

ASPECT	BEFORE (FILE-BASED)	AFTER (IW AI CORE)
ID allocation	Markdown append (race conditions)	DB `FOR UPDATE` (atomic)
State tracking	JSON manifests in ai-dev/work/	PostgreSQL tables
Dashboard data	File scanning + mtime polling	DB queries + SSE
Multi-project	Impossible (hardcoded to InnoForge)	First-class (project_id everywhere)
Skills distribution	Copied manually	`iw sync-skills` (versioned)
Recovery actions	Manual manifest editing	One-click from dashboard
Analytics	None	DB queries over historical data
Daemon location	Inside InnoForge repo	Standalone iw-ai-core repo
Completed work items	Accumulate in `ai-dev/design/done/` forever (git bloat)	Tier 1 (DB: design docs, reports) + Tier 2 (archive: compressed artifacts)
Content search	grep across files	PostgreSQL full-text search + future RAG (pgvector)
Design doc viewing	Open file from filesystem	Always-instant from DB, any item, any age
Artifact storage	Loose files in git (permanent bloat)	`.tar.zst` on disk (not in git), on-demand extraction

Appendix B: Port Assignments

SERVICE	PORT	PURPOSE
iw-ai-core PostgreSQL	5433	Platform database (separate from app DBs)
iw-ai-core Dashboard	9900	Unified web UI
InnoForge PostgreSQL	5432	Application database
InnoForge API	8000	Application API
InnoForge Frontend	5173/5174	Application UI